AI Foundations II: AI-Accelerated Research

Best practices for AI-powered research workflows, from setup to analysis.

Nicholas Warren

Lead, Artificial Intelligence & Machine Learning program, DISSC

nick.warren@yale.edu

Session objectives

By the end of this session you will be able to:

- Write effective prompts that produce useful R code
- Use Claude Code to explore concepts, build analyses, iterate on output, and debug
- Critically evaluate AI-generated code and statistical claims
- Apply best practices for AI-powered research workflows

What is Claude Code?

Claude Code is a tool that lets you talk to Claude directly from the terminal in VS Code. Unlike ChatGPT or other web-based assistants, Claude Code runs where you work (inside the container) and can see your project files.

It can:

- **Read your project files.** No need to copy and paste code into a chat window.
- **Edit files directly** when you ask it to make changes
- **Run code** like R scripts on your behalf
- **Answer questions** about your code, data, and analysis in plain language

Open the workshop notebook

- In the **Explorer** (top icon in the Activity Bar on the left), click `example.qmd`
- This is a **Quarto notebook**: a document that mixes written text with blocks of R code (similar to R Markdown). Each block of code is called a **code chunk**.
- Click the **Run Cell** button (the play icon ▶️ that appears above each code chunk) to run it
- Output (plots, tables, text) appears directly below each chunk

> For extra prompt ideas throughout the session, see `prompts/examples.md` in the Explorer.

Start Claude Code

Open a **new terminal** in VS Code (`Ctrl + ` ` or `Terminal > New Terminal`). Type:

```
claude
```

This starts a conversation with Claude. You type messages and Claude responds. To start a new session later, just type `claude` again in the terminal. Useful commands:

Command	Action	Command	Action
`/help`	Show commands	`/model`	Switch models
`/clear`	Clear history	`Ctrl + C`	Cancel response
`/exit`	Exit Claude Code	`Ctrl + D`	Exit Claude Code

Select the right model

Once Claude Code is running, type `/model` and select **Opus 4.7** from the list. Use the **up/down arrow keys** to choose the model, then use the **left/right arrow keys** to set the thinking level to **max**. Press `Enter` to confirm.

This gives you the most capable model with the highest-quality reasoning. If you skip this step, Claude Code may default to a less capable model.

> **Tip:** If Claude's responses start to feel unfocused during a long session, type `/compact` to summarize the conversation and free up its attention.

Two terminals, two tools

VS Code's bottom Panel can have multiple terminals open at once. In this workshop you'll use two:

- **R Terminal**: runs R code. Use it for `library()`, running code chunks, and interactive R work. Click `R: (not attached)` in the bottom status bar (the very bottom edge of the VS Code window) to start it.
- **Terminal**: a general-purpose terminal running inside the container (zsh on both macOS and Windows hosts, since the container is Linux). This is where you run `claude` to start Claude Code.

To switch between them, click the **dropdown arrow** next to the terminal name at the top of the bottom panel (it might say "R Interactive" or "zsh"). You can also click the **+** icon to open a new terminal.

Tips for working with Claude Code

- **Be specific.** "Add clustered standard errors at the state level" works better than "improve the model."
- **Point to files.** Claude Code can read your project, but referencing a file by name (e.g., "in example.qmd") focuses its attention.
- **Iterate.** If the first result isn't right, follow up: "That's close, but use `sandwich` instead of `estimatr`."
- **Ask it to explain.** "Why did you use that approach?" is always a good follow-up.

Review before you accept

Claude Code will ask for permission before editing files or running commands. Before approving:

- **Read the proposed changes.** Claude Code highlights exactly what it plans to add, remove, or modify. Make sure you understand the edit before accepting.
- **Run the code yourself.** After Claude edits a chunk, run it to confirm it produces the output you expect.
- **Verify statistical claims.** AI can make reasoning errors. Check coefficients, p-values, and interpretations against your own understanding.

Container safety

The Docker container protects your computer in several ways:

- **Limited internet access.** The container can only connect to approved services (Claude's AI service, GitHub). It cannot browse the web or reach other sites.
- **File access restrictions.** Claude can only see and edit files in the `workspace/` folder. Your personal files, email, and everything else on your computer are completely inaccessible.
- **Blocked dangerous commands.** Settings inside the container prevent Claude from running commands that could delete important files or change system settings.

Skipping permission prompts

Because the container is sandboxed, you can optionally start Claude Code with:

```
claude --dangerously-skip-permissions
```

This skips the approval prompts that appear before each action, so Claude can work faster. Despite the alarming name, this is safe inside the container. Claude still cannot access anything outside `workspace/`.

Claude Memory

Claude Code can **remember things across sessions**. When you correct it, share a preference, or teach it something about your project, it can save that as a memory and recall it next time.

- **User memories:** your role, preferences, and expertise level
- **Project memories:** context about your project files and ongoing work
- **Feedback memories:** corrections you've given (e.g., "don't use that package")

Type `/memory` in Claude Code to see what it has stored and toggle memory on or off.

Where memory lives

Claude Code runs inside the container, so its memory is stored inside the container too. It's never uploaded or shared.

Location (inside the container)	What it contains
`~/.claude/CLAUDE.md`	Your personal instructions for all projects in this sandbox
`.claude/CLAUDE.md`	Project instructions (inside `workspace/`, visible to you on the host)
`~/.claude/projects/<name>/memory/`	Notes Claude writes from your conversations

> `~/` inside the container is a Docker volume, separate from your macOS or Windows home folder. It persists across rebuilds but only for *this* sandbox.

Inspecting memory from the host

Anything under `workspace/` is visible on your host machine and shows up in the VS Code Explorer:

- **macOS:** `<your-project>/workspace/.claude/`
- **Windows:** `<your-project>\workspace\.claude\`

Personal memories (`~/.claude/...`) live in the container's volume. To read them, open the **Terminal** in VS Code (inside the container) and run `cat ~/.claude/CLAUDE.md` — or just ask Claude: *"show me what's in your personal memory."*

Skills

Skills are shortcuts you can type in Claude Code to perform common actions. They all start with `/`.

Skill	What it does
`/memory`	View and manage what Claude remembers
`/model`	Switch between available Claude models
`/simplify`	Ask Claude to review and clean up recent code changes
`/init`	Generate a starting `CLAUDE.md` instructions file for your project

Custom skills

You can create your own skills by adding a `SKILL.md` file to your project. A skill is just a text file with instructions that Claude follows when you invoke it.

- **Personal skills:** stored at `~/.claude/skills/` *inside the container* — available in any session of this sandbox
- **Project skills:** stored at `.claude/skills/` inside `workspace/` — committed alongside your code so anyone who clones the project gets them

This workshop includes a `/modernize` skill that converts old-style R code to modern tidyverse style. Open `.claude/skills/modernize/SKILL.md` in the Explorer to see how it works, then try it: type `/modernize` in Claude Code.

> Full docs: https://code.claude.com/docs/en/skills

Hands-on exercises

We'll work through a real dataset using four phases: explore the research question, write code, iterate on the analysis, and debug. Open `example.qmd` and make sure Claude Code is running.

Phase 1: Concept & Idea Generation

Use Claude Code to explore the research question **before writing any code**.

Try these prompts:

> What is Difference-in-Differences and when is it appropriate to use?

> I have organ donation rate data for 27 states across 6 quarters. California introduced an "active choice" policy in Q3 2011. How would I set up a DiD analysis?

Phase 2: Script Creation

Ask Claude Code to write the import code. It can edit the notebook directly.

> Read in data/organ_donations.csv and prepare it for a DiD analysis. Write the code in the Phase 2 section of example.qmd.

Run the chunks Claude creates. You should see 162 rows with columns including `treated_state` (California vs. Control) and `post` (TRUE/FALSE). Phase 3's code depends on these exact names.

Phase 2: About the data

The dataset (`data/organ_donations.csv`) has 162 rows: 27 states across 6 quarters (Q4 2010 through Q1 2012). California's "active choice" organ donation policy took effect in Q3 2011.

Data source: Kessler & Roth (2014), "Don't Take 'No' For An Answer." Dataset via the `causaldata` R package, featured in *The Effect* by Nick Huntington-Klein.

Phase 3: Analysis & Visualization

The notebook has pre-written code for a DiD plot and regression model. Run these chunks, then use Claude Code to iterate:

> Change the y-axis to show values from 0% to 100%

> Switch the plot to a black-and-white theme suitable for print

> Add robust standard errors clustered at the state level

> Create a summary statistics table broken down by treatment group and period

Phase 3: More ideas

> Export the coefficient table as a CSV and the plot as a high-resolution PNG to an output/ folder

> Add a placebo test using a fake treatment date one quarter earlier

> Explain the coefficients from the DiD regression in plain language

See `prompts/examples.md` for more prompts.

Phase 4: Debugging

The bottom of the notebook has four code chunks with intentional bugs. Ask Claude Code to find and fix them:

> Look at the Phase 4 section of example.qmd and fix the bugs

Then try a follow-up:

> For each bug you fixed, explain what went wrong and how a beginner could spot the same mistake

Best practices

- **Start broad, then narrow.** Begin with conceptual questions ("What is DiD?"), then move to specific code requests ("Add clustered standard errors").
- **One task at a time.** Shorter, focused requests produce better results than long, multi-part ones.
- **Save your work.** Before making major changes, duplicate your notebook as a backup. For a more robust approach, ask Claude: "Help me set up version control so I can track changes and undo mistakes." It will walk you through the process.
- **Trust but verify.** Claude is a powerful tool, not a replacement for your domain expertise. Always review its output critically.

Troubleshooting

- **Container won't build:** Make sure Docker Desktop is running. On Windows, confirm WSL 2 is installed (Foundations I, Step 3). Try `Cmd/Ctrl+Shift+P` > "Rebuild Container Without Cache."
- **Claude Code not responding:** Check that your credentials are set (Foundations I, Step 3). Close VS Code and reopen it to pick up your API key.

Troubleshooting (continued)

- **R packages not found:** R packages are pre-installed in the container. Try rebuilding: `Cmd/Ctrl+Shift+P` > "Rebuild Container Without Cache."
- **No code completion:** The `languageserver` package is pre-installed. Try restarting VS Code.
- **Files missing:** Verify you're looking in the `workspace/` folder. Claude can only see files there.

Next steps

- **Get long-term access.** Your workshop API key will expire. Contact dissc@yale.edu for ongoing access.
- **Try your own data.** Copy a dataset into `workspace/` and ask Claude Code to help you analyze it.
- **One project per sandbox.** Run `npm create @yale-dissc/agent-sandbox@latest new-project` for each new project.
- **Explore further.** Try creating scripts, debugging errors, or explaining unfamiliar code.
- **Get in touch.** nick.warren@yale.edu or dissc@yale.edu

Thank you

This workshop was developed with contributions from:

- **Maurice Dalton** and the Data Engineering and Solution Team, DISSC
- **Data-Intensive Social Science Center (DISSC)**
- **Yale University Library**

And thank you to all participants whose questions and feedback continue to shape these materials.